Variable Font Inspector — Technical Documentation

Oct 23, 2026 · @Syed Jafri

Overview

Variable Font Inspector is a specimen and audit tool: load any variable (or static) font, type arbitrary sample text, and see how axes, kerning and OpenType features behave across the whole design space — with machine-readable audit reports alongside the visual specimen.

Who it is for — type designers checking a build before release, font engineers in CI, and developers or designers evaluating a font for a product.

Key capabilities

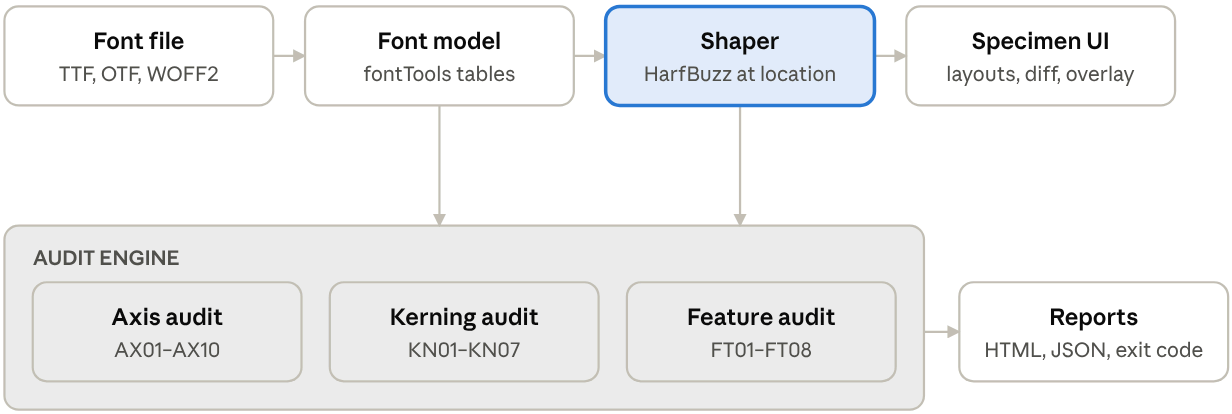
- Axis inspection: every `fvar` axis and named instance, `avar` mapping curves, `STAT` naming, and interpolation sweeps at configurable steps.
- Kerning audit: extract all effective pairs (GPOS PairPos format 1 and 2, legacy `kern`), compare across axis locations, flag missing, zero, outlier and collision pairs.
- Feature coverage: list every GSUB/GPOS feature per script/language, test which glyphs each feature actually touches for the sample text, and report gaps against a target feature set.
- Specimen: live multi-row rendering at chosen weights/instances with toggleable features, waterfalls, and diff views between two locations or two font versions.
- Export: HTML/PDF specimen, JSON/CSV audit reports, and a non-zero CLI exit code for CI gating.

Out of scope — font editing or fixing, hinting/TrueType instruction auditing, and color font (COLR/CPAL, SVG) inspection in v1.

Architecture

A single parsed font model and a single shaping path serve both the visual specimen and the audits, so what the audit reports is exactly what the specimen shows.

One font model and one shaper feed both the specimen and every audit



system architecture · model, shaper, specimen, audits

The axis audit reads table data straight from the model; kerning and feature audits shape text through HarfBuzz at each sampled location. Stack: Python 3.11 backend (fontTools, uharfbuzz, FastAPI) for the CLI, CI and server; a TypeScript/React web client using harfbuzzjs (WASM) and CSS `font-variation-settings` for live rendering. Shaping results are cached by (font hash, location, features, text).

Font loading and table parsing

Fonts are parsed once into a normalized in-memory model; all audits read that model, and all rendering goes through HarfBuzz so results match real shaping.

Accepted inputs — TTF, OTF (CFF/CFF2), WOFF, WOFF2, and TTC/OTC collections (each face selectable). Max file size 50 MB by default.

Table	What is extracted	Used by
`fvar`	Axes (tag, min/default/max, name ID, hidden flag), named instances	Axis audit, UI sliders
`avar` (v1/v2)	Segment maps per axis	Mapping curves, normalization
`STAT`	Axis records, axis values, elided fallback name	Naming checks
`gvar` / `CFF2`	Which glyphs vary, tuple regions	Interpolation sweep

Table	What is extracted	Used by
`HVAR` / `MVAR`	Advance and metrics variation	Spacing at locations
`GPOS`	Scripts, languages, features, lookups (PairPos, MarkBase, Cursive…)	Kerning, feature coverage
`GSUB`	Scripts, languages, features, lookups	Feature coverage
`kern`	Legacy format 0 pairs	Kerning (fallback)
`cmap`, `name`, `OS/2`, `hmtx`	Character map, names, weight class, advances	Coverage, labels

Coordinate handling — user-space values are converted to normalized coordinates (−1…0…1) via `fvar` then `avar`; the model stores both so reports can quote either.

Libraries — fontTools (parsing, instancing, varLib model) on the backend; HarfBuzz (via `uharfbuzz` / `harfbuzzjs`) for shaping; FreeType or the browser's font engine for raster output.

Variable axis auditing

The axis audit samples the design space on a grid and checks that metadata, naming and interpolated outlines are consistent at every sample.

Sampling — for each axis, sample min, default, max, every named-instance coordinate, and `axes.steps` evenly spaced points (default 9). Multi-axis fonts use the full grid up to `axes.max_samples` (default 400), then fall back to Latin-hypercube sampling plus all corners.

Checks

ID	Check	Severity
AX01	Registered axes (`wght`, `wdth`, `opsz`, `ital`, `slnt`) within spec ranges (e.g. `wght` 1–1000, `slnt` −90–90)	error
AX02	Default value lies within min/max and matches a named instance	warn

ID	Check	Severity
AX03	Named instance coordinates inside axis ranges; no duplicate coordinates or names	error
AX04	`OS/2.usWeightClass` matches default `wght`	warn
AX05	`STAT` present, covers every `fvar` axis, has an elided fallback name	warn
AX06	`avar` maps are monotonic and hit −1, 0, 1	error
AX07	Stem/advance width changes monotonically along `wght` / `wdth` for test glyphs (default `H O n o`)	warn
AX08	No outline self-intersection or point-order flip at sampled locations	error
AX09	Bounding boxes stay within `hhea` ascender/descender at all locations	warn
AX10	Hidden axes flagged; registered-axis tags lowercase, custom tags uppercase	info

Outputs — a per-axis report with the `avar` curve, a stem-width-vs-axis plot for each test glyph, and a list of failing locations, each linkable to the specimen at those exact coordinates.

Kerning pair auditing

Kerning is measured by shaping, not by reading tables alone: the effective adjustment for a pair is the advance difference between shaping the pair with `kern` on and off.

Pair sources

1. **Table pairs** — every pair defined in GPOS PairPos (format 1 glyph pairs; format 2 class pairs expanded to glyphs, capped at `kerning.max_expand`, default 250 000) and legacy `kern`.
2. **Sample-text pairs** — every adjacent glyph pair produced by shaping the user's sample text (after GSUB, so ligatures and alternates are included).
3. **Reference lists** — bundled critical pair lists (e.g. `AV To Ty LT P. r, "A f)` and Latin/Cyrillic/Greek core sets), extendable with user CSVs.

Measurement — for each pair at each sampled axis location: kern value (font units and em-relative), and the minimum horizontal gap between outlines after kerning, measured on rasterized or flattened contours at `kerning.probe_rows` heights.

Flags

ID	Flag	Rule
KN01	Missing	Reference pair has no kerning and its gap is < `kerning.min_gap_em` × UPM or > 1.5× the median space
KN02	Collision	Outlines overlap or gap < 0 after kerning
KN03	Outlier	Kern value beyond ±3 MAD of its class's pairs
KN04	Zero / redundant	Pair entry with value 0, or class kern overridden to the same value
KN05	Interpolation drift	Gap at an intermediate location falls outside the range of the masters' gaps by > 2% em
KN06	Asymmetry	Mirrored pair (e.g. `(` `[` vs `]` `)`) differs by > `kerning.symmetry_tol`
KN07	Exception conflict	Glyph-pair exception reverses the sign of its class kern

Views — a sortable pair table (filter by flag, glyph, class, location), a kerning matrix heatmap for chosen glyph sets, and a string view that highlights flagged pairs inside the sample text.

OpenType feature coverage auditing

Coverage answers two questions: which features does the font declare, and which of them actually change the sample text at each location.

Declared inventory — for every script and language system in GSUB and GPOS: feature tags, lookup indices, lookup types and flags. `FeatureVariations` (e.g. `rvrn` swaps along `wght`) are evaluated at each sampled location.

Effective coverage — the sample text is shaped once with defaults, then once per feature toggled on (or off for default-on features such as `liga`, `kern`, `calt`). A feature counts as *effective* for the text if glyph IDs, clusters or positions differ. The report records which characters and glyphs each feature touched.

Glyph-level reach — independent of sample text, each lookup's coverage table is resolved to glyphs and Unicode values, giving "what *could* this feature affect" versus "what it *did* affect".

Checks

ID	Check	Severity
FT01	Target profile features missing (profiles: `latin-basic`, `latin-pro`, `cyrillic`, `greek`, `arabic`, custom YAML)	error
FT02	Feature declared but no lookups, or lookups with empty coverage	warn
FT03	Feature registered under `DFLT` but missing for a script the `cmap` supports	warn
FT04	Figure-set features (`lnum` `onum` `pnum` `tnum`) inconsistent — e.g. `tnum` advances not equal	error
FT05	`smcp` / `c2sc` cover fewer letters than the font's lowercase/uppercase set	warn
FT06	Stylistic sets `ss01`–`ss20` lack `featureParams` UI names	info
FT07	Feature result differs between axis locations without a `FeatureVariations` reason	warn
FT08	Characters in sample text missing from `cmap` (.notdef)	error

Views — a feature × script matrix, a toggle panel that re-renders the specimen live, and a before/after glyph strip for each feature.

Specimen UI, sample text and export

The specimen is a grid of rows (locations) × sample strings, all shaped by HarfBuzz and rendered at a common size so differences stay visible.

Sample text sources

- Free text typed by the user (any Unicode, with script/language/direction auto-detected and overridable).
- Built-in sets: pangrams per script, kerning strings, figure tables, diacritic stacks, punctuation and case-sensitive forms.
- Generated strings: every glyph in `cmap` ordered by Unicode or by glyph ID; "adhesion"-style strings built from a chosen character set.

Layouts

Layout	Shows
Waterfall	One string at multiple sizes and one location
Axis ramp	One string at N steps along one axis
Instance grid	All named instances × chosen strings
2-axis matrix	e.g. `wght` × `wdth`, one cell per sample
Diff	Two locations or two font builds overlaid (onion skin) or side by side, with changed glyphs highlighted
Glyph grid	All glyphs with names, Unicode, advance, and variation status

Controls — per-axis sliders and numeric inputs (user or normalized units), named-instance picker, feature toggles, size, tracking, line height, script/language override, and an *audit overlay* that marks flagged kerning pairs and .notdef in place.

Shareable state — the full view (font hash, coordinates, features, text, layout) serializes to a URL fragment and a `.vfi.json` session file.

Export — HTML specimen (self-contained, font embedded as WOFF2 subset), PDF (vector, via headless Chromium), PNG per row, and JSON/CSV audit reports.

Configuration, CLI and API

The same audit engine runs in the web UI, the `vfi` CLI and the Python API; a YAML config fixes sample text, locations and thresholds so results are reproducible in CI.

`vfi.yaml`

```
font: build/MySans[wdth,wght].ttf
compare_to: release/MySans-1.2[wdth,wght].ttf   # optional, enables diff
sample_text:
  - "Hamburgefonstiv 0123456789"
  - file: samples/kerning-latin.txt
script: latn
language: dflt
axes:
  steps: 9
  max_samples: 400
  locations:                      # extra explicit locations
    - {wght: 350, wdth: 87.5}
  test_glyphs: [H, O, n, o]
kerning:
  reference_lists: [latin-core, punctuation]
  max_expand: 250000
  min_gap_em: 0.02
  probe_rows: 32
  symmetry_tol: 5                 # font units
features:
  profile: latin-pro
  require: [ss01, case]
report:
  formats: [html, json]
  fail_on: error                  # error | warn | never
```

CLI

```
vfi inspect MySans.ttf                       # print axes, instances,
features
vfi audit   --config vfi.yaml                # run all audits
vfi audit   --config vfi.yaml --only kerning --loc wght=700
vfi specimen MySans.ttf --text "Type here" --layout axis-ramp --axis wght --
out ramp.pdf
vfi diff    old.ttf new.ttf --config vfi.yaml --out diff.html
vfi serve   --port 8080                      # local web UI
```

Exit codes: 0 pass, 1 findings at or above `fail_on`, 2 font could not be parsed, 3 config error.

Report JSON (one finding)

```
{"id": "KN02", "severity": "error", "audit": "kerning",
 "location": {"wght": 900, "wdth": 75},
 "glyphs": ["T", "o"], "kern_units": -140, "min_gap_em": -0.008,
 "message": "Outlines collide after kerning"}
```

Python API

```
from vfi import Font, Audit
f = Font.open("MySans[wdth,wght].ttf")
print(f.axes)                              # [Axis('wght', 100, 400, 900),
...]
res = Audit(f, config="vfi.yaml").run()
res.findings(severity="error").to_csv("errors.csv")
f.shape("office", loc={"wght": 700}, features={"liga": False})
```

Testing, limitations and roadmap

Audits are tested against fonts with known, deliberately planted defects so every check ID has a failing and a passing fixture.

Testing

- Unit tests per check ID using small generated fonts (fontTools `FontBuilder`) with one defect each.
- Golden reports for a fixed corpus of open-licence variable fonts (OFL), compared field by field on every change.
- Shaping parity: CLI (uharfbuzz) and web (harfbuzzjs) outputs must match glyph IDs and positions exactly for the corpus.
- Visual regression of specimen PNGs with a per-pixel tolerance.
- Performance budget: full audit of a 2-axis, 1 000-glyph font in under 30 s on a 4-core machine; specimen slider updates under 50 ms.

Known limitations

- Collision detection is geometric on flattened outlines; it ignores hinting and sub-pixel rendering differences.

- Class-kern expansion is capped; very large class kerning is sampled, and the report says so.
- Feature effect is judged on the supplied text; a feature can be correct but appear "ineffective" if the text never triggers it — the glyph-level reach view covers this.
- Complex-script shaping (Indic, Arabic joining) is audited for coverage only, not for correctness of forms.
- Browser rendering can differ slightly from FreeType; PDF export uses the browser engine.

Roadmap

- [ ] COLRv1 and color palette inspection
- [ ] `avar2` cross-axis mapping visualisation
- [ ] Spacing (sidebearing) audit alongside kerning
- [ ] GitHub Action wrapper posting audit summaries on pull requests
- [ ] Language-specific feature checks via `locl` and CLDR exemplar sets